



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

IoT Challenge 1 - Wokwi and Power Consumption

TESI DI LAUREA MAGISTRALE IN
TELECOMMUNICATION ENGINEERING

Author: **Hong CHEN**

Student ID: [REDACTED]

Advisor: Prof. Matteo Cesana

Co-advisors: Alessandro Redondi

Academic Year: 2025-03

Contents

Contents	i
1 IoT Challenge 1	1
1.1 System Design	1
1.2 Software Implementation	2
1.3 Figures and Data Analysis	3
1.3.1 Energy Consumption Analysis (Before and After Optimization) . .	4
1.3.2 Presentation of Simulation Results	8

1 | IoT Challenge 1

1.1. System Design

The core of the project, ESP32 handles data processing and device communication.

- **Wi-Fi:** Connects the device to the internet via Wi-Fi for data transmission.
- **HC-SR04:** Measures the distance between an object and the sensor.
- **Transmitter Module:** This module transmits the sensor-collected data to a remote server or device via Wi-Fi.
- **ESP-NOW Protocol:** A low-power wireless communication protocol for device-to-device data transfer.

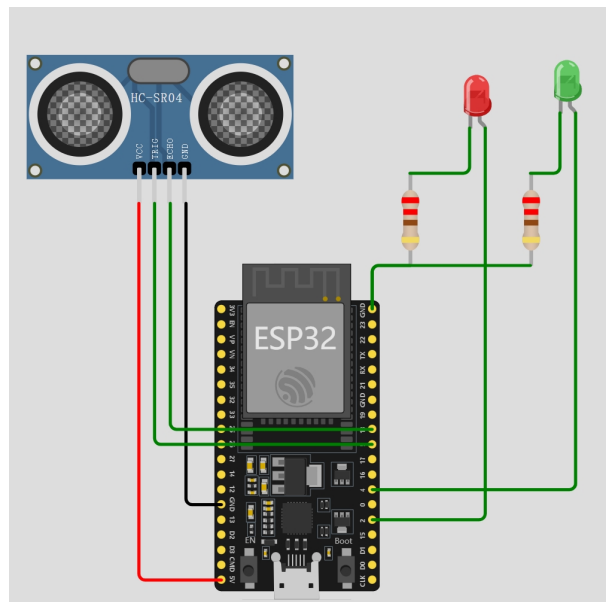


Figure 1.1: ESP32 Model

1.2. Software Implementation

The code uses the ESP32 board, an ultrasonic sensor (HC-SR04), and the ESP-NOW protocol to implement a parking space occupancy detection system.

- (1) **Sensor Acquisition:** Use ultrasonic sensors to measure distance and determine parking space occupancy status. The 10s high-level pulse on the TRIG_PIN is only used to notify the sensor to start emitting ultrasound. After the trigger is completed, the state of the TRIG_PIN is irrelevant to subsequent measurements.

```
duration = pulseIn(ECHO_PIN, HIGH); //
        ↳Measure the duration of the high-level
        ↳pulse
```

When the ultrasound returns, the sensor outputs a high-level pulse on the ECHO_PIN, and the width of the pulse is proportional to the distance of the obstacle. The pulseIn() function waits for ECHO_PIN to go HIGH and records the duration of the high-level pulse (in microseconds).

The calculation follows the speed of sound principle:

$$\text{Distance} = \frac{\text{Duration} \times \text{Speed of Sound}}{2} \quad (1.1)$$

Where the speed of sound is assumed to be 343 m/s.

- (2) **WiFi Radio Management:**

WiFi.mode(WIFI_STA), sets the ESP32 to station mode.

esp_now_init() initializes the ESP-NOW protocol.

esp_now_add_peer() configures and adds a peer node.

Broadcast data via ESP-NOW, and turn off WiFi after transmission to avoid idle listening (P_WIFI_OFF = 308mW vs P_WIFI_ON = 777mW).

```
WiFi.disconnect(true); // Force disconnect
WiFi.mode(WIFI_OFF);    // Completely turn off
        ↳the radio hardware
```

- (3) **Data Transmission:** The code starts by sending data over ESP-NOW using the esp_now_send() function. The parameters passed to this function include the destination MAC address (broadcastMac), the data to be sent (data), and the size of the data (sizeof(data)). The transmission process includes a delay mechanism implemented through a while loop. In this loop, the program waits until the transmission

is marked as complete (TX_DONE), or until a timeout occurs.

- (4) **Data Processing:** Encapsulate the distance data into a 5-byte message (1 byte status + 4 bytes floating-point number).
- (5) **Energy Consumption Calculation:** Track the operating time and power consumption of each module to estimate battery life. Total energy consumption per cycle can be calculated as:

$$E_{\text{total}} = \sum (P_{\text{module}} \times t_{\text{module run time}})$$

- (6) **Deep Sleep Control:**

```
esp_sleep_enable_timer_wakeup(SLEEP_TIME * 1000);
    ↪ // Timer-based wakeup (unit: microseconds)
esp_deep_sleep_start();
    ↪ // Enter deep sleep mode (P_SLEEP = 60mW)
```

During sleep, the MCU and RF are powered off, reducing power consumption from 777mW (WiFi active) to 60mW. The timer manages periodic wake-ups to ensure that the parking sensor remains functional without compromising energy efficiency.

1.3. Figures and Data Analysis

The power consumption, duration, and energy in different states are shown in the table below.

Table 1.1: Power Consumption and Duration in Different States

State	Power Consumption (mW)	Duration (ms)	Energy Consumption (mJ)
BOOT	313	1	0.31
Sensor Reading	350	4.439	1.75
Wi-Fi ON	777	263	204.35
Transmission	1220	0.942	1.15
Wi-Fi OFF	308	1.340	0.41
Active Cycle	784.81	265	207.97
Deep Sleep	60	41000	2460.00
Total Energy per Cycle	64.65	41265.00	2667.97

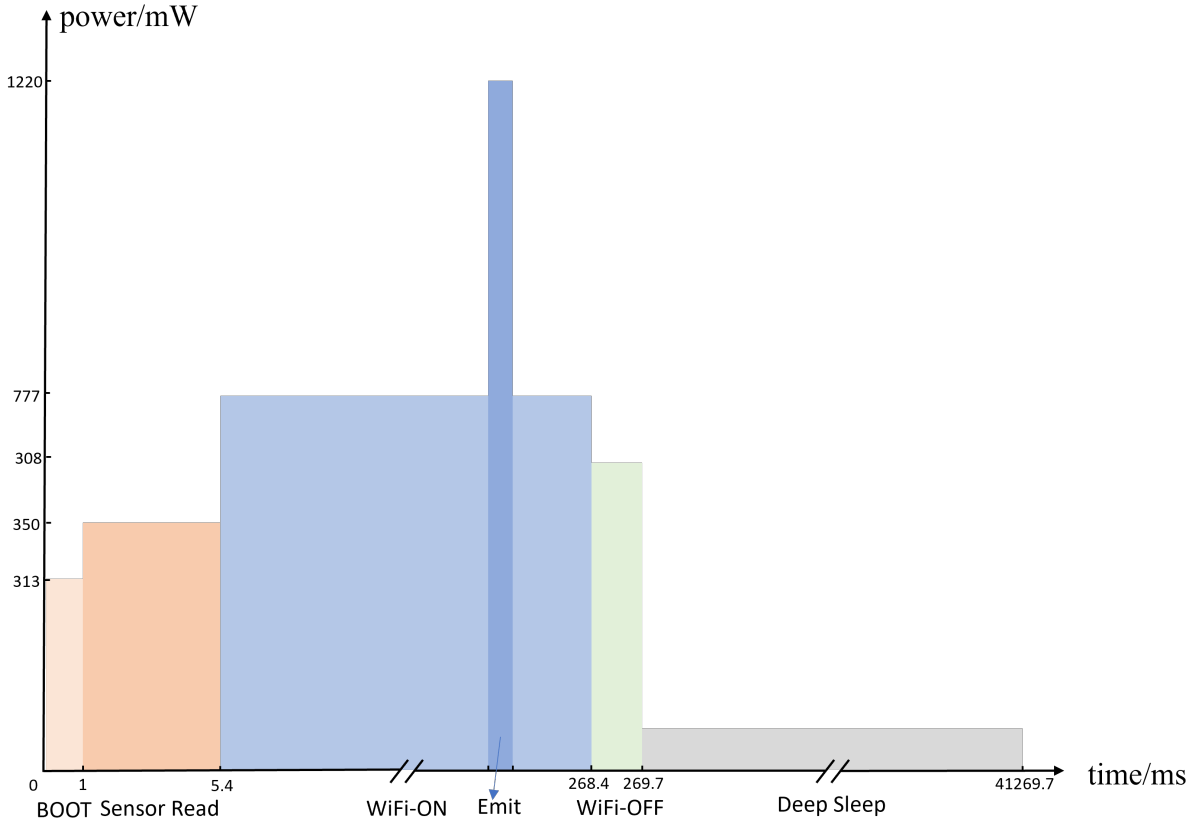


Figure 1.2: Power VS. time in one cycle

1.3.1. Energy Consumption Analysis (Before and After Optimization)

As shown in Figure 1.4, the histogram represents the energy consumption proportion of different states. The chart shows the proportion of energy consumed by each state (Wi-Fi ON, sensor, transmission, deep sleep). From this, we can see that the Deep Sleep state generally consumes the most energy, while the other states consume the least.

Since the majority of energy consumption is attributed to the deep sleep state, the proportion of energy consumption from other states is relatively small, making it hard to observe clearly. Therefore, in the following comparisons, the deep sleep state is omitted.

Figure 1.5 and 1.6 shows the bar chart of energy consumption in different states before and after optimization.

Figure 1.8 and 1.7 shows the time proportion of different states in one cycle.

In the initial (pre-optimization) setup, the Wi-Fi module remained active for longer periods, leading to higher energy consumption during idle or non-communication periods.

Energy Consumption: Deep Sleep vs. Other States

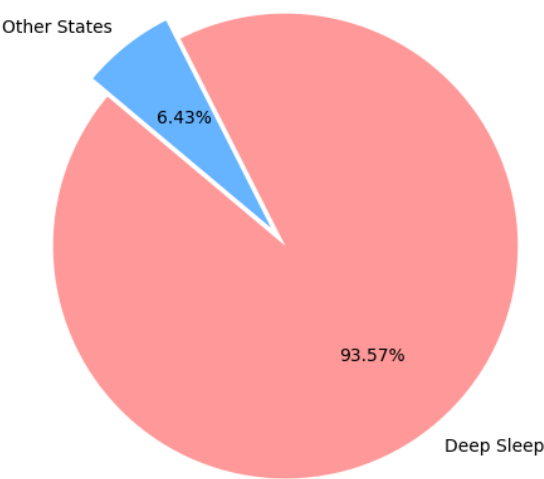


Figure 1.3: The pie chart of Energy Consumption

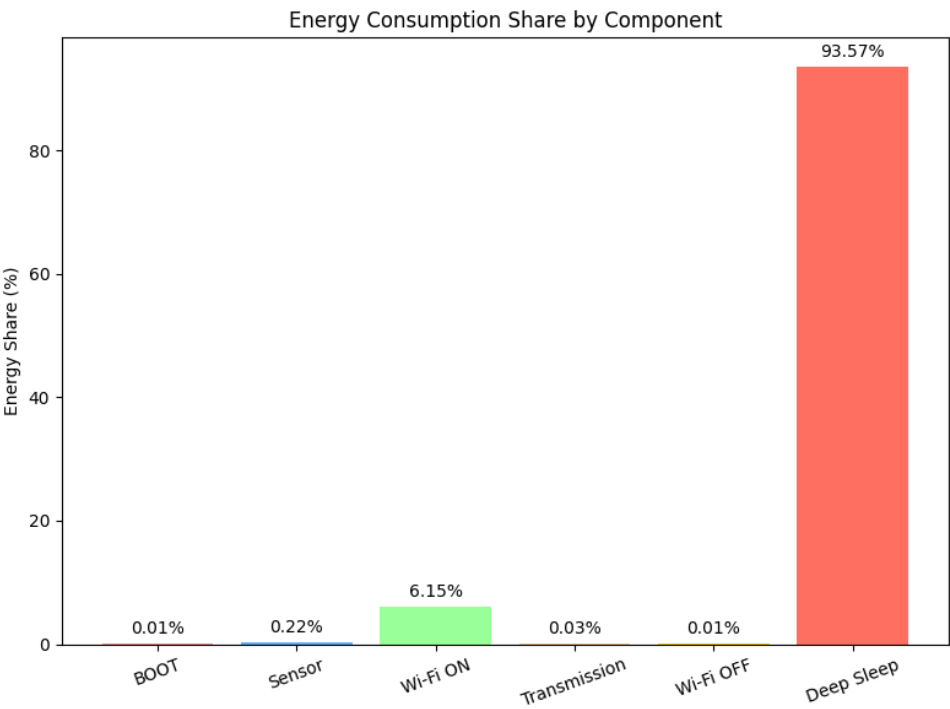


Figure 1.4: Energy Consumption

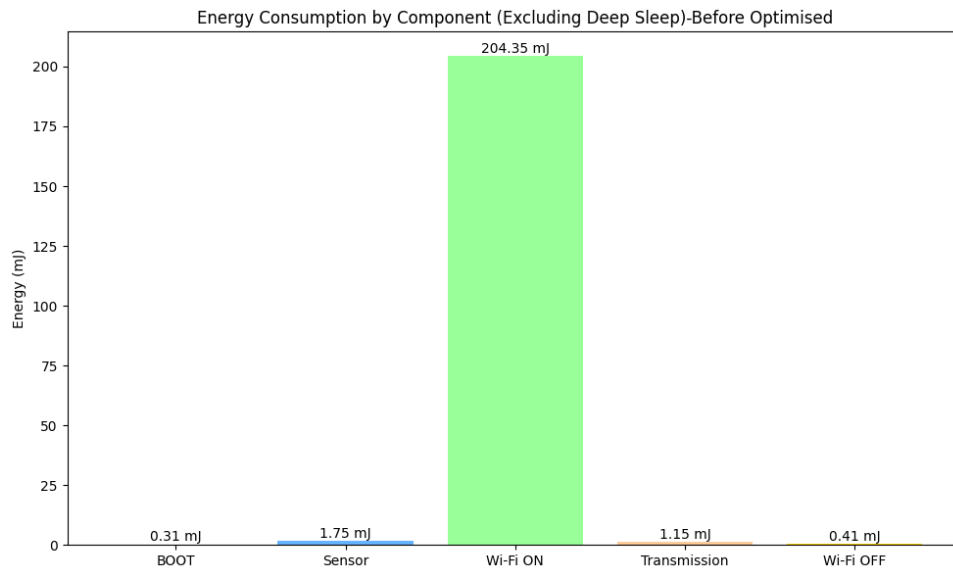


Figure 1.5: Energy Consumption (Active Period)

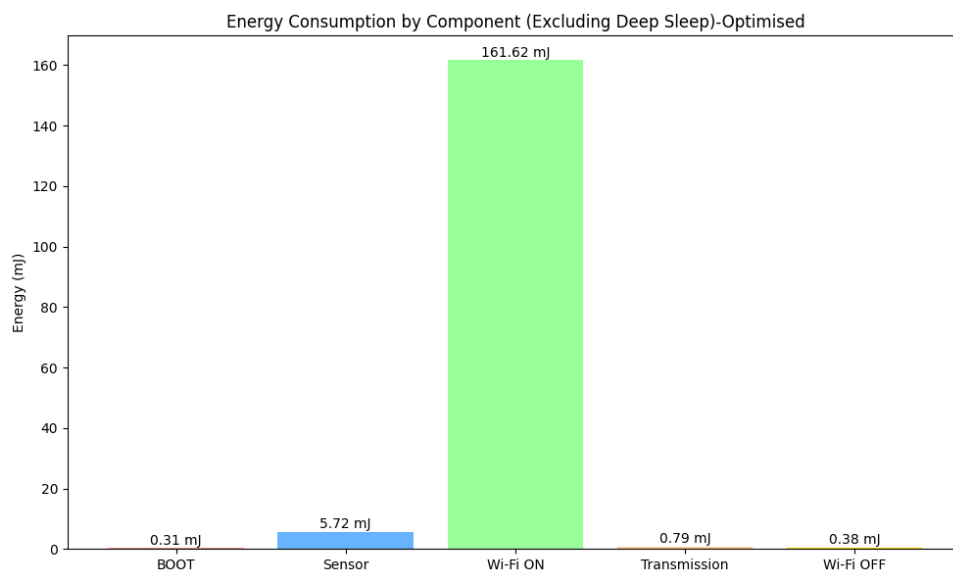


Figure 1.6: The Energy Consumption (Active Period)-Optimised

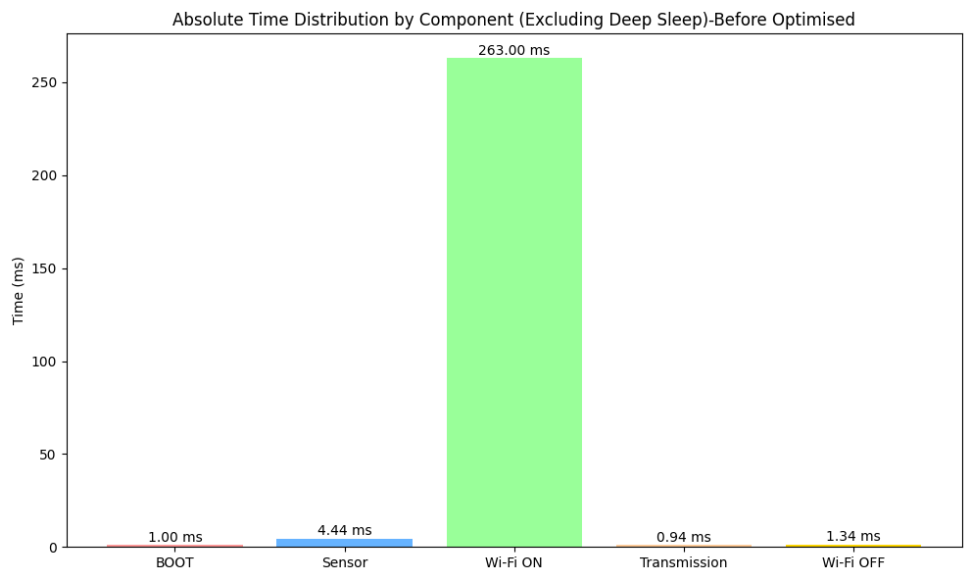


Figure 1.7: The Time Duration (Active Period)

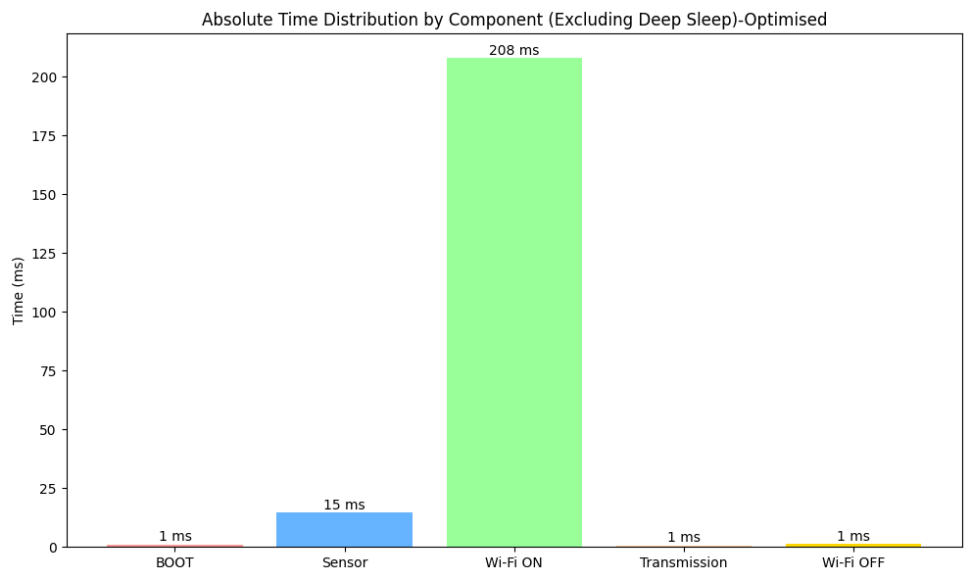


Figure 1.8: The Time Duration (Active Period)-Optimised

In IoT devices, enabling the Wi-Fi module often leads to higher power consumption, especially when the Wi-Fi is turned on unnecessarily. Therefore, the objective of this optimization is to limit the Wi-Fi module activation to after the sensor successfully measures and validates the data, thereby reducing unnecessary power consumption and extending the device's battery life.

From Figure 1.5 , it is evident that the energy consumption of the Wi-Fi ON state is higher than that of the other states. By optimizing the Wi-Fi ON time and reducing the idle duration when Wi-Fi is on, the overall energy consumption can be effectively reduced, as shown in Figure 1.6.

In the original code, the Wi-Fi module was activated regardless of whether the sensor's measurement was valid. Even when invalid data was returned by the sensor, Wi-Fi was turned on, leading to unnecessary power consumption.

After the data is sent, Wi-Fi is immediately turned off to maximize battery energy savings.

```
duration = pulseIn(ECHO_PIN, HIGH);
WiFi.mode(WIFI_STA);
```

As shown in Figure 1.7 and 1.8, the duration of the Wi-Fi ON state after optimization is significantly reduced compared to before. By strictly controlling the duration of Wi-Fi ON, the time spent in this state is reduced to 79% of its original value.

Table 1.2: Comparison of Final Data

	Pre-Optimisation	After Optimisation
WiFi-ON Duration (ms)	263	208
WiFi-ON Energy Consumption (mJ)	204.35	161.62
Batt. Life: (hours)	85.22	86.41

1.3.2. Presentation of Simulation Results

Figure 1.10 shows the specific test results. The LED indicates the response of the receiver after receiving the data, with the green light representing an available parking space. The detailed operation log and results are shown in Figure 1.10 (b).

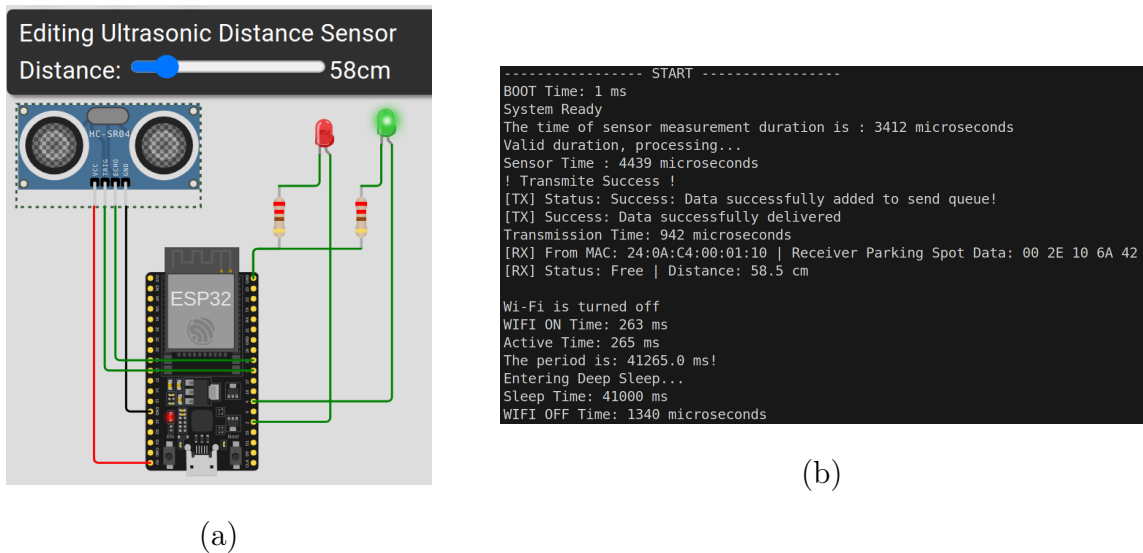


Figure 1.9: (a): ESP32 Testing Process
(b): Actual Running Output Result

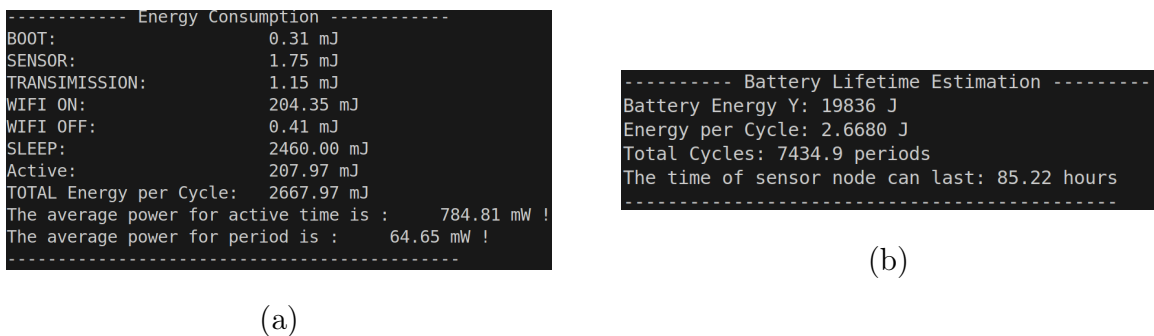


Figure 1.10: (a): Result of Energy Consumption
(b): Device Lifetime